

```
import matplotlib.pyplot as plt
from mlxtend.plotting import plot_decision_regions
from sklearn import datasets
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Dense
import numpy as np
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from keras.callbacks import ModelCheckpoint
from keras.regularizers import L2
```

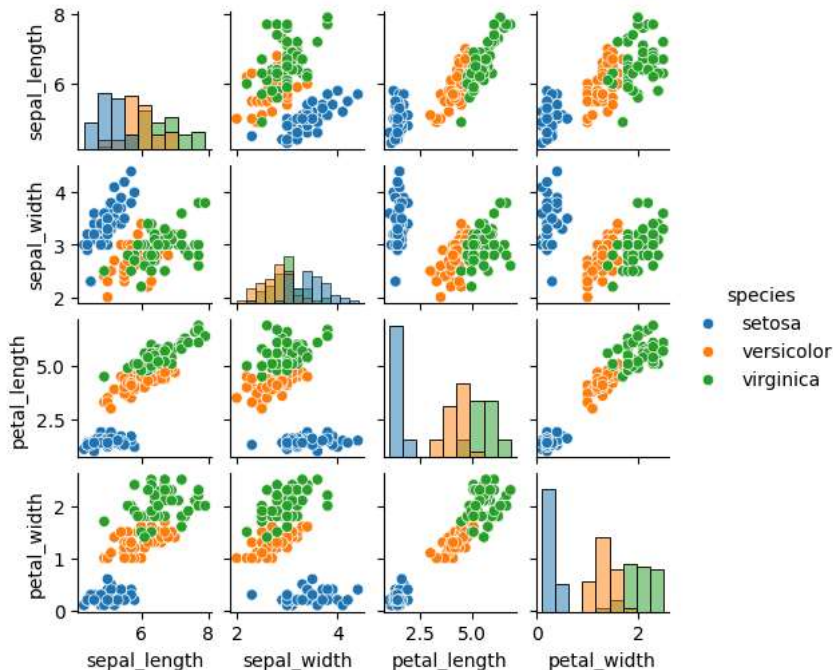
```
iris_sns=sns.load_dataset("iris")
```

```
iris_sns.groupby(['species']).count()
```

	sepal_length	sepal_width	petal_length	petal_width
species				
setosa	50	50	50	50
versicolor	50	50	50	50
virginica	50	50	50	50

```
sns.pairplot(iris_sns, hue="species", diag_kind="hist",height=1.3)
```

```
<seaborn.axisgrid.PairGrid at 0x7d10af809d90>
```



```
# Feature matrix Scaling:
```

```
X = iris_sns[["sepal_length","petal_length"]]
```

```
#Create a scaler model that is fit on the input data.
```

```
scaler = StandardScaler().fit(X)
```

```
#Scale the numeric feature variables
```

```
X = scaler.transform(X)
```

```
# Target Encoding
```

```
from sklearn import preprocessing
```

```
#initialize the encoder
```

```
label_encoder = preprocessing.LabelEncoder()
```

```
#fit the encoder to our targetdata
```

```

iris_sns['species'] = label_encoder.fit_transform(iris_sns['species'])

#y is now composed of 3 distinct integers: 0,1,2
y=iris_sns[['species']]

# Splitting the data and performing Onehot encoding
from keras.utils import to_categorical

X_train,X_test,y_train,y_test=train_test_split(X,y)
y_train_onehot=to_categorical(y_train)
y_test_onehot=to_categorical(y_test)

# Build the model and fit it to Iris data
checkpoint_filepath = '/tmp/checkpoint.weights.h5' # Changed the filepath to include .weights.h5
model_checkpoint_callback = ModelCheckpoint(
    filepath=checkpoint_filepath,
    save_weights_only=True,
    monitor='val_accuracy',
    mode='max',
    save_best_only=True)

lambdaa=0
model = Sequential()
model.add(Dense(256,input_shape=(2,),activation='tanh',name='hidden_layer',kernel_initializer='he_uniform',
    kernel_regularizer=L2(lambdaa)))
#model.add(Dense(64,activation='tanh',name='hidden_layer2',kernel_initializer='glorot_uniform',kernel_regularizer=L2(lambdaa)))
model.add(Dense(3,activation='softmax',name='output_layer'))

model.compile(optimizer="adam",loss="CategoricalCrossentropy",metrics=["accuracy"])
history=model.fit(X_train,y_train_onehot,epochs=30,verbose=1,validation_data=(X_test,y_test_onehot),
    callbacks=[model_checkpoint_callback])

```

```

4/4 ————— 0s 63ms/step - accuracy: 0.7824 - loss: 0.7044 - val_accuracy: 0.8158 - val_loss: 0.5651
Epoch 3/30
4/4 ————— 0s 37ms/step - accuracy: 0.8131 - loss: 0.5597 - val_accuracy: 0.8158 - val_loss: 0.4989
Epoch 4/30
4/4 ————— 0s 39ms/step - accuracy: 0.7841 - loss: 0.5130 - val_accuracy: 0.7632 - val_loss: 0.4771
Epoch 5/30
4/4 ————— 0s 45ms/step - accuracy: 0.7763 - loss: 0.4713 - val_accuracy: 0.7632 - val_loss: 0.4584
Epoch 6/30
4/4 ————— 0s 34ms/step - accuracy: 0.7930 - loss: 0.4520 - val_accuracy: 0.7632 - val_loss: 0.4425
Epoch 7/30
4/4 ————— 0s 35ms/step - accuracy: 0.8003 - loss: 0.4231 - val_accuracy: 0.7895 - val_loss: 0.4305
Epoch 8/30
4/4 ————— 0s 46ms/step - accuracy: 0.8424 - loss: 0.3933 - val_accuracy: 0.7895 - val_loss: 0.4209
Epoch 9/30
4/4 ————— 0s 35ms/step - accuracy: 0.7606 - loss: 0.4494 - val_accuracy: 0.8158 - val_loss: 0.4062
Epoch 10/30
4/4 ————— 0s 38ms/step - accuracy: 0.8004 - loss: 0.4072 - val_accuracy: 0.8158 - val_loss: 0.3941
Epoch 11/30
4/4 ————— 0s 41ms/step - accuracy: 0.8280 - loss: 0.3858 - val_accuracy: 0.8158 - val_loss: 0.3844
Epoch 12/30
4/4 ————— 0s 35ms/step - accuracy: 0.8415 - loss: 0.3715 - val_accuracy: 0.8158 - val_loss: 0.3751
Epoch 13/30
4/4 ————— 0s 35ms/step - accuracy: 0.8190 - loss: 0.3980 - val_accuracy: 0.8158 - val_loss: 0.3663
Epoch 14/30
4/4 ————— 0s 50ms/step - accuracy: 0.8482 - loss: 0.3364 - val_accuracy: 0.8158 - val_loss: 0.3591
Epoch 15/30
4/4 ————— 0s 39ms/step - accuracy: 0.8524 - loss: 0.3440 - val_accuracy: 0.8158 - val_loss: 0.3519
Epoch 16/30
4/4 ————— 0s 39ms/step - accuracy: 0.8393 - loss: 0.3525 - val_accuracy: 0.8158 - val_loss: 0.3452
Epoch 17/30
4/4 ————— 0s 32ms/step - accuracy: 0.8695 - loss: 0.3109 - val_accuracy: 0.8158 - val_loss: 0.3398
Epoch 18/30
4/4 ————— 0s 41ms/step - accuracy: 0.8427 - loss: 0.3399 - val_accuracy: 0.8421 - val_loss: 0.3313
Epoch 19/30
4/4 ————— 0s 29ms/step - accuracy: 0.8978 - loss: 0.2953 - val_accuracy: 0.8158 - val_loss: 0.3274
Epoch 20/30
4/4 ————— 0s 26ms/step - accuracy: 0.8905 - loss: 0.3072 - val_accuracy: 0.8421 - val_loss: 0.3206
Epoch 21/30
4/4 ————— 0s 29ms/step - accuracy: 0.9232 - loss: 0.2803 - val_accuracy: 0.8421 - val_loss: 0.3152
Epoch 22/30
4/4 ————— 0s 31ms/step - accuracy: 0.9362 - loss: 0.2611 - val_accuracy: 0.8421 - val_loss: 0.3098
Epoch 23/30
4/4 ————— 0s 34ms/step - accuracy: 0.9199 - loss: 0.2956 - val_accuracy: 0.8684 - val_loss: 0.3013

```

```

4/4 ————— 0s 27ms/step - accuracy: 0.9458 - loss: 0.2554 - val_accuracy: 0.8421 - val_loss: 0.2800
Epoch 27/30
4/4 ————— 0s 43ms/step - accuracy: 0.9354 - loss: 0.2653 - val_accuracy: 0.8421 - val_loss: 0.2804
Epoch 28/30
4/4 ————— 0s 28ms/step - accuracy: 0.9448 - loss: 0.2360 - val_accuracy: 0.8684 - val_loss: 0.2749
Epoch 29/30
4/4 ————— 0s 26ms/step - accuracy: 0.9499 - loss: 0.2462 - val_accuracy: 0.8684 - val_loss: 0.2696
Epoch 30/30
4/4 ————— 0s 25ms/step - accuracy: 0.9655 - loss: 0.2184 - val_accuracy: 0.8684 - val_loss: 0.2678

```

```
# "mlxtend" for decision boundary visualization
```

```
class Onehot2Int(object):
```

```
    def __init__(self, model):
        self.model = model
```

```
    def predict(self, X):
        y_pred = self.model.predict(X)
        return np.argmax(y_pred, axis=1)
```

```
# Wrap keras model
model_no_ohe = Onehot2Int(model)
```

```
# Plot decision boundary
plot_decision_regions(np.array(X_train), np.array(y_train['species']), clf=model_no_ohe, legend=2)
plt.show()
```

9600/9600 ————— 19s 2ms/step

